Smart Outcome Predictor — Final Report

Ensemble-based prediction of course completion (classification) and final score (regression)

1. Executive summary

- Ensemble methods improve markedly over a single decision tree: ROC-AUC 0.712 → 0.788 (classification) and RMSE 11.57 → 10.12 (regression).
- Best ensemble by 5-fold cross-validation: **XGBoost (tuned)** (classification) and **XGBoost (tuned)** (regression); both refit on all 5,200 students and deployed in the Streamlit app.
- Test performance of the deployed models — Accuracy 0.742, F1 0.629, ROC-AUC 0.788; MAE 8.13, RMSE 10.12, R^2 0.437.
- A regularised linear benchmark (Logistic Regression ROC-AUC 0.794; Ridge RMSE 10.04) is statistically tied with the best ensemble, because the signal is mostly linear and the data are noisy.
- Predictions are driven by assessment and engagement: top features are avg_quiz_score, attendance_rate, time_spent_hours (classification) and avg_quiz_score, attendance_rate, time_spent_hours (regression).

2. Data and method

5,200 students, 15 predictors (10 numeric, 5 categorical) after dropping *student_id*, *course_start_date* and the opposite target (to prevent leakage). 80/20 stratified split (random_state = 42). Median imputation + standard scaling for numeric and one-hot encoding for categorical features, wrapped in a scikit-learn *Pipeline* with every model. Models were compared on the test set and with 5-fold cross-validation (robustness); the deployed model was chosen by CV score, never by test score.

3. Results

Classification (test set, sorted by ROC-AUC)

Model	Acc	Prec	Rec	F1	ROC-AUC	CV AUC	CV std
Logistic Regression	0.745	0.687	0.590	0.634	0.794	0.785	0.009
AdaBoost	0.733	0.685	0.531	0.598	0.789	0.774	0.009
XGBoost (tuned)	0.742	0.684	0.582	0.629	0.788	0.780	0.009
Gradient Boosting	0.739	0.676	0.582	0.625	0.788	0.771	0.008
Stacking	0.739	0.685	0.559	0.616	0.787	0.779	0.011
Voting (soft)	0.729	0.682	0.518	0.589	0.785	0.776	0.010
XGBoost	0.731	0.660	0.582	0.619	0.780	0.771	0.007
Bagging	0.725	0.657	0.559	0.604	0.777	0.770	0.012
Decision Tree (single)	0.679	0.580	0.520	0.549	0.712	0.696	0.015
Voting (hard)	0.728	0.712	0.462	0.560	0.675	–	–

Hard voting has no probabilities; its ROC-AUC is computed from class labels and is not comparable.

Regression (test set, sorted by RMSE)

Model	MAE	RMSE	R^2	CV RMSE	CV std
Ridge	8.084	10.045	0.446	9.536	0.124
Stacking	8.124	10.121	0.437	9.654	0.119
XGBoost (tuned)	8.131	10.123	0.437	9.626	0.118
Voting	8.165	10.124	0.437	9.736	0.132
XGBoost	8.155	10.170	0.432	9.708	0.146
Gradient Boosting	8.207	10.200	0.428	9.701	0.120
Bagging	8.229	10.241	0.424	9.846	0.082
AdaBoost	8.606	10.572	0.386	10.268	0.115
Decision Tree (single)	9.239	11.566	0.265	11.469	0.166

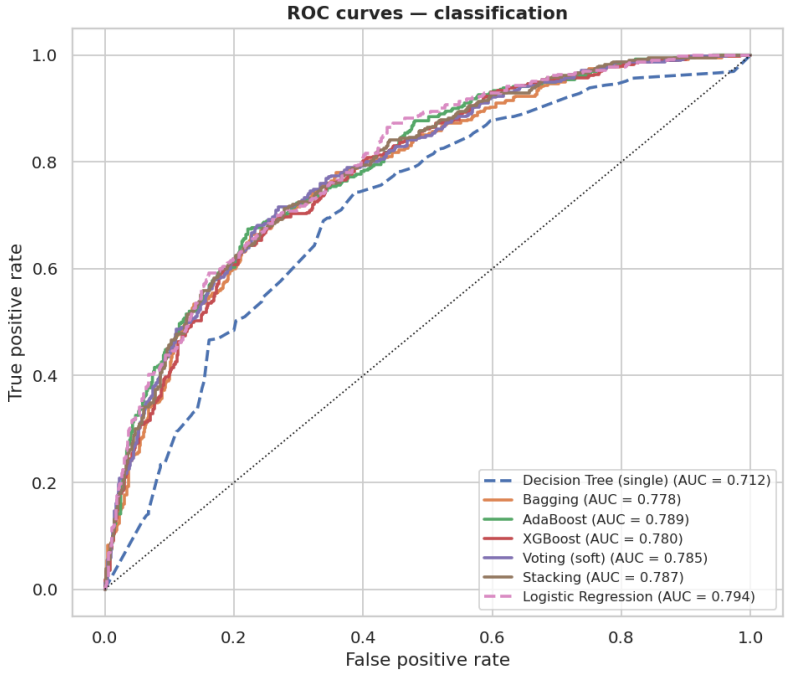


Figure 1 — ROC curves of the main ensembles and baselines.

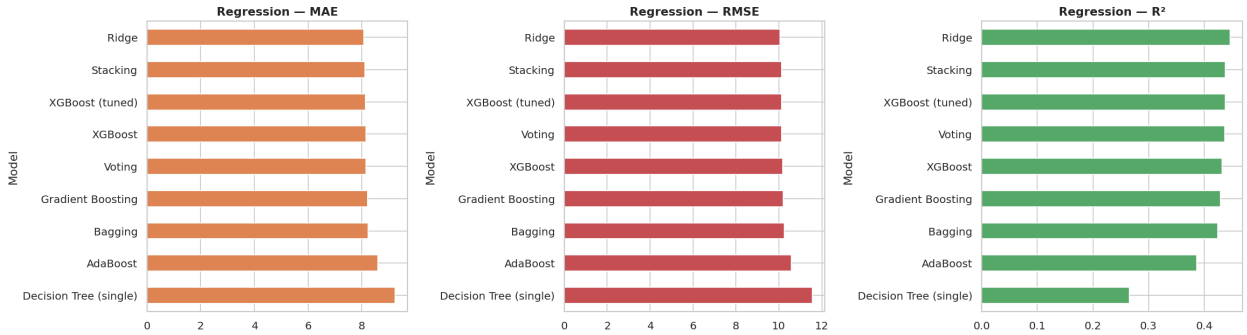


Figure 2 — Regression error metrics for all models.

4. Impact of Bagging vs Boosting

Model	Accuracy	F1	ROC-AUC	Train ROC-AUC	Gap (train−test)
Decision Tree (single)	0.679	0.549	0.712	0.884	0.172
Bagging	0.725	0.604	0.777	0.938	0.160
AdaBoost	0.733	0.598	0.789	0.800	0.011
XGBoost	0.731	0.619	0.780	0.859	0.079

Model	MAE	RMSE	R^2	Train RMSE	Gap (test−train)
Decision Tree (single)	9.239	11.566	0.265	8.418	3.148
Bagging	8.229	10.241	0.424	7.696	2.546
AdaBoost	8.606	10.572	0.386	10.007	0.565
XGBoost	8.155	10.170	0.432	8.556	1.615

- **Bagging** lifts the single tree by reducing variance (ROC-AUC and RMSE improve markedly) but its deep trees still over-fit the training data (large train–test gap).
- **Boosting** (AdaBoost, XGBoost) builds shallow learners sequentially, achieving similar or better test scores with a much smaller over-fitting gap; XGBoost is also the fastest to train.
- Gains are real but bounded by irreducible noise in the data.

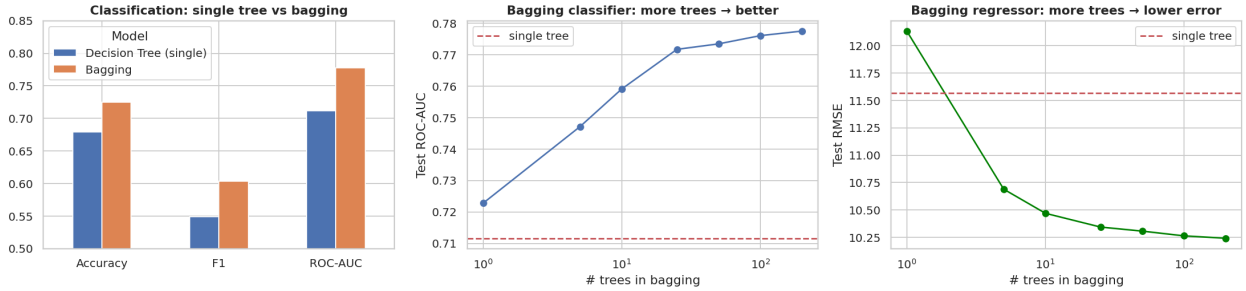


Figure 3 — Single tree vs bagging, and effect of the number of bagged trees.

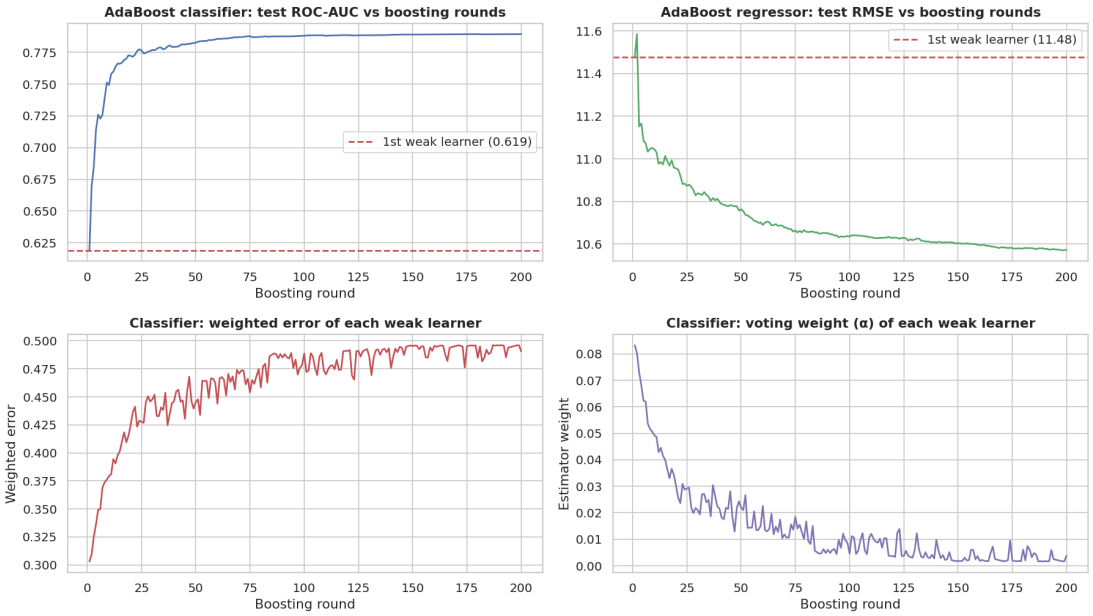


Figure 4 — AdaBoost: sequential improvement of the weak learners.

5. Comparison of tree-based boosting algorithms

Algorithm	Observed behaviour
AdaBoost	Test ROC-AUC 0.789 / RMSE 10.57. Very small over-fitting gap; clearly weakest on regression; sensitive to noise.
Gradient Boosting	ROC-AUC 0.788 / RMSE 10.20. Accurate and stable, slower to train.
XGBoost	ROC-AUC 0.780 / RMSE 10.17. Fast, regularised, stable across CV folds.
XGBoost (tuned)	ROC-AUC 0.788 / RMSE 10.12. Randomised-search tuning gave a modest improvement in cross-validated score.

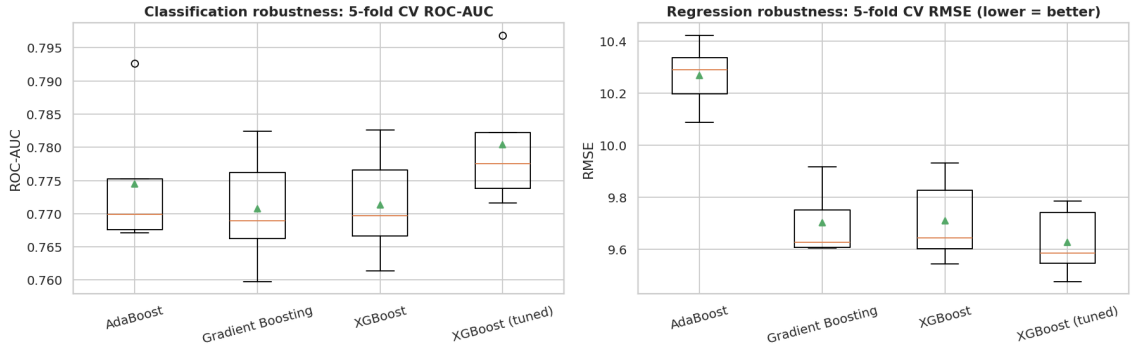


Figure 5 — Robustness of the boosting models across 5 CV folds.

6. Advantages of Voting and Stacking

- **Soft voting** (ROC-AUC 0.785) clearly outperforms **hard voting** (F1 0.560 vs 0.589) because it uses model confidence — but it did not beat its best member here, because the members' errors are strongly correlated.
- **Stacking** (ROC-AUC 0.787; RMSE 10.12) learns how to weight its base models and is among the top performers, at the price of heavier training/inference.
- Both are most useful when the base models are diverse (linear, tree-based, instance-based, boosted).

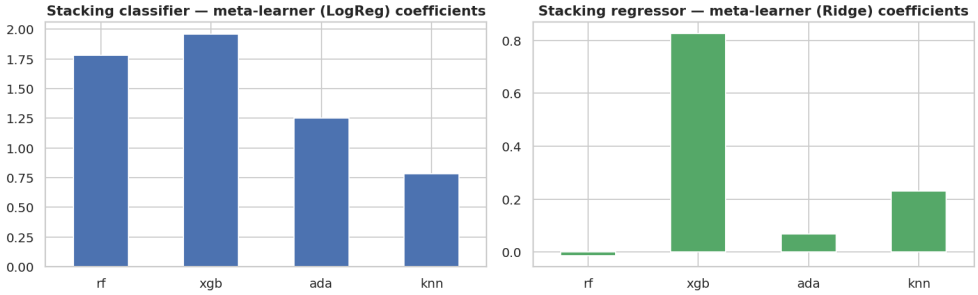


Figure 6 — Coefficients learned by the stacking meta-learners.

7. Final model recommendation for deployment

Classifier: XGBoost (tuned). Regressor: XGBoost (tuned). Selected by best 5-fold cross-validated ROC-AUC / RMSE among all ensemble techniques, then refit on the full dataset. They offer top-tier accuracy, small variance across folds, sub-second training, native handling of non-linearities and missing values, and a simple deployment (single scikit-learn pipeline saved with joblib).

Honest note: the linear benchmark (Logistic Regression / Ridge) performs essentially the same on this dataset. If interpretability or maximal simplicity is the priority, it is a perfectly valid production alternative; the ensemble is recommended here because it is robust to non-linear patterns, outliers and future data drift.

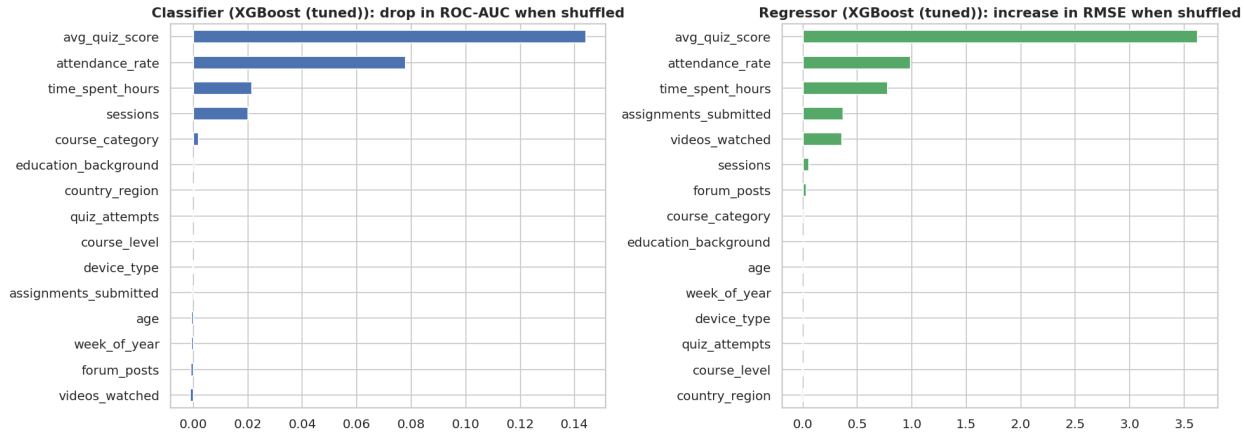


Figure 7 — Permutation importance of the deployed models.

8. Business interpretation, limitations and next steps

- Use the **completion probability** to rank students by drop-out risk and trigger early support (reminders, mentoring) — especially for low quiz scores and low attendance in the first weeks.
- Use the **predicted final score** to flag likely under-performers and offer targeted practice.
- Recall for completers at the default 0.5 threshold is moderate; set the threshold from the relative cost of missing an at-risk student vs an unnecessary intervention (the app has a threshold slider).
- **Limitations:** modest signal (ROC-AUC ≈ 0.79, R^2 ≈ 0.44); many true scores are capped at 100; data appear synthetic; single hold-out split + CV; no fairness audit yet.
- **Next steps:** week-by-week activity features, probability calibration, threshold optimisation, fairness analysis across regions/education levels, drift monitoring after deployment.